什么是树

> **树是无环连通的图，二叉树指每个父节点最多有两个子节点的树**

定义二叉树： 数据对象：具有相同特性的数据元素集合。

数据关系：若为空，则没有结点；若非空，则有唯一根结点，其余结点分成左子树和右子树两个互不相交的集合。

二叉树存储

> **可以用顺序表，也可以用链表**

顺序表二叉树

#include <stdio.h> #include <stdlib.h> #include <stdbool.h>

#define MAX_TREE_SIZE 100 /*

- 顺序存储二叉树结点
- - data：结点数据
- - used：当前位置是否有结点 */ typedef struct { int data; bool used; } SeqTreeNode;

/*

- 顺序存储二叉树
- - nodes：结点数组
- - size：数组最大容量 */ typedef struct { SeqTreeNode nodes[MAX_TREE_SIZE+1]; int size; } SeqBiTree;

void init_tree(SeqBiTree *tree){ if(tree==NULL){ return; } tree->size=0; for(int i=0;i<=MAX_TREE_SIZE;i++){ tree->nodes[i].used = false; } }

bool set_root(SeqBiTree *tree, int value){ if(tree==NULL){ return false; } if(tree->nodes[1].used == true){ return false; } tree->nodes[1].used=true; tree->nodes[1].data=value; tree->size++; return true; } bool set_left_child(SeqBiTree tree, int parent_node, int value){ if(tree==NULL){ return false; } if(parent_node2>MAX_TREE_SIZE){ return false; } if(tree->nodes[parent_node*2].used == true){ return false; } tree->nodes[parent_node2].used=true; tree->nodes[parent_node*2].data=value; if(parent_node2>tree->size){ tree->size=parent_node2; } return true; } bool set_right_child(SeqBiTree *tree, int parent_node, int value){ if(tree==NULL){ return false; } if(parent_node*2+1>MAX_TREE_SIZE){ return false; } if(tree->nodes[parent_node2+1].used == true){ return false; } tree->nodes[parent_node*2+1].used=true; tree->nodes[parent_node2+1].data=value; if(parent_node*2+1>tree->size){ tree->size=parent_node*2+1; }; return true; } void level_order(SeqBiTree *tree){ if(tree==NULL){ return; } for(int i=1,j=1;i<=tree->size;i++){ if(tree->nodes[i].used==true){ printf("%d ",tree->nodes[i].data); } else{ printf("-1 "); } if(i==j){ printf("\n"); j=2*j+1; } } } int

main(){ SeqBiTree tree; init_tree(&tree); set_root(&tree, 1); set_left_child(&tree, 1, 2); set_right_child(&tree, 1, 3); set_left_child(&tree, 2, 4); set_right_child(&tree, 3, 7); level_order(&tree); return 0; }

链表二叉树和递归

#include <stdio.h> #include <stdlib.h>

typedef struct TreeNode { int data; struct TreeNode *left; struct TreeNode *right; } TreeNode;

TreeNode* create_node(int value) { TreeNode* newnode = (TreeNode*)malloc(sizeof(TreeNode)); if (newnode == NULL) { return NULL; } newnode->data = value; newnode->left = NULL; newnode->right = NULL; return newnode; }

void preorder(TreeNode *root) { if (root == NULL) return; printf("%d ", root->data); preorder(root->left); preorder(root->right); }

void inorder(TreeNode *root) { if (root == NULL) return; inorder(root->left); printf("%d ", root->data); inorder(root->right); }

void postorder(TreeNode *root) { if (root == NULL) return; postorder(root->left); postorder(root->right); printf("%d ", root->data); }

int depth(TreeNode *root) { if (root == NULL) return 0; int left = depth(root->left); int right = depth(root->right); return (left > right ? left : right) + 1; }

int main() { TreeNode *root = create_node(1); root->left = create_node(2); root->right = create_node(3); root->left->left = create_node(4); root->left->right = create_node(5); root->right->left = create_node(6); root->right->right = create_node(7);

```
printf("前序遍历: ");
preorder(root);
printf("\n");

printf("中序遍历: ");
inorder(root);
printf("\n");

printf("后序遍历: ");
postorder(root);
printf("\n");

printf("树的深度: %d\n", depth(root));

return 0;
```

}

递归本质上是函数不断的嵌套调用自己，内层会对外层有影响； 循环是反复执行一段程序，不同程序之间是平行关系

递归先序

#include <stdio.h> #include <stdlib.h>

typedef struct TreeNode { int data; // 节点存储的数据 struct TreeNode *left; // 左子树指针 struct TreeNode *right; // 右子树指针 } TreeNode;

TreeNode* create_node(int value) { TreeNode* newnode = (TreeNode*)malloc(sizeof(TreeNode)); if (newnode == NULL) { return NULL; } newnode->data = value; newnode->left = NULL; newnode->right = NULL; return newnode; }

typedef struct Stack { //栈的定义 TreeNode **arr; //存储数组的指针 int top; //栈顶下标 int capacity; //容量 } Stack;

Stack *createStack(int capacity) { //创造栈 Stack *stack = malloc(sizeof(Stack)); stack->arr = malloc(sizeof(TreeNode *) * capacity); stack->top = -1; stack->capacity = capacity; return stack; }

int isEmpty(Stack *stack) { //判断栈是否为空 return stack->top == -1; }

void push(Stack *stack, TreeNode *node) { //把节点压到栈顶 if (stack->top == stack->capacity - 1) { return; } stack->arr[++stack->top] = node; }

TreeNode *pop(Stack *stack) { //弹出栈顶节点 if (isEmpty(stack)) { return NULL; } return stack->arr[stack->top--]; }

void preorderTraversal(TreeNode *root){ *Stack* stack=createStack(100); push(stack, root); while(!isEmpty(stack)){ TreeNode* node=pop(stack); printf("%d ",node->data); if(node->left!=NULL){ preorderTraversal(node->left); } if(node->right!=NULL){ preorderTraversal(node->right); } } free(stack->arr); free(stack); }

int main() { TreeNode *root = create_node(1); root->left = create_node(2); root->right = create_node(3); root->left->left = create_node(4); root->left->right = create_node(5); root->right->left = create_node(6); root->right->right = create_node(7);

```
preorderTraversal(root);

return 0;
```

}